



NAME : HAMDAN ALI

WEEK 5 : Day 21

DAY 21

Create a React App and render a simple heading using JSX

Task : Screenshoots Of Output

CODEBASE Of the Work

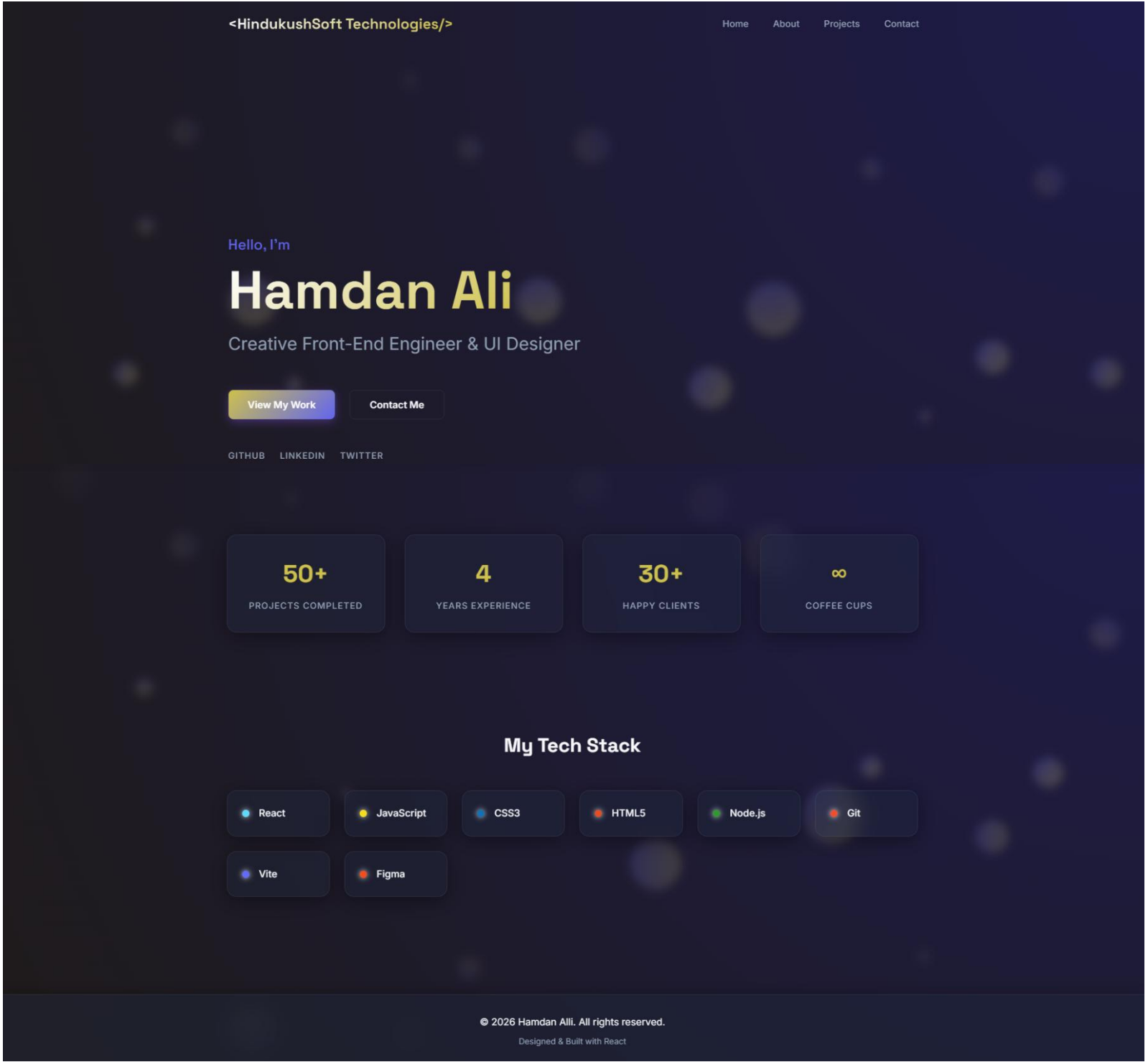
```
EXPLORER
└─ INTERNSHIP ( HINDUKUSHSOFT TECHNOLOGIES )
  └─ Week1
  └─ Week2
  └─ Week3
  └─ Week4
  └─ Week5
    └─ Day21
      └─ node_modules
      └─ src
        └─ App.css
        └─ App.jsx
        └─ index.css
        └─ main.jsx
        └─ index.html
        └─ package-lock.json
        └─ package.json
        └─ Task21.doc
        └─ vite.config.js
      └─ Day22
      └─ Day23
      └─ Day24
      └─ Day25

index.html ...Day19
React Heading — Day 21
JS script.js ...Day19
index.html ...Day21

Week5 > Day21 > src > App.jsx > App
4  const App = () => {
43  };
44
45  // Generate particles
46  const particles = Array.from({ length: 20 }).map((_, i) => (
47    <div key={i} className={`particle p-${i}`}></div>
48  ));
49
50  return (
51    <div className="portfolio-container">
52      <div className="particles-container">
53        {particles}
54      </div>
55
56      <nav className={`navbar ${navScrolled ? 'scrolled' : ''}`>
57        <div className="nav-content">
58          <div className="logo">&lt;HindukushSoft Technologies&gt;</div>
59          <ul className="nav-links">
60            {navLinks.map((link, index) => (
61              <li key={index}><a href={`#${link.toLowerCase()}`}>{link}</a></li>
62            ))}
63          </ul>
64        </div>
65      </nav>
66
67      <main className="main-content">
68        <section id="home" className="hero-section">
69          <div className="hero-content">
70            <h2 className="greeting">Hello, I'm</h2>
71            <h1 className="name-title">
72              <span className="gradient-text">Hamdan Ali</span>
```



Full Task View





EXPLANATION OF THE PROJECT

DAY 21 — Create a React App and render a simple heading using JSX

Day 21: React App with JSX Heading

Author: Alex (Frontend Intern)

Date: August 13

Introduction and Approach

Today I built a beautiful Developer Portfolio Landing Page to really show off what I've learned about JSX and React. I wanted this to feel premium, so I went with a dark theme, glassmorphism effects, and some really cool CSS animations like a floating particle background and a typewriter effect for the subtitle.

"JSX is just fantastic. It looks like HTML, but it lets me use the full power of JavaScript right inside my markup. I can map over arrays to generate UI, use variables for dynamic content, and conditionally render things effortlessly."

Setup and Tools Used

Instead of relying on boilerplate creators like Create React App or `npm create vite`, I built the entire project structure from scratch to understand how the pieces fit together:

- **package.json:** Set up my dependencies (React, ReactDOM) and dev dependencies (Vite, Vite React plugin).
- **vite.config.js:** Kept it simple with the React plugin enabled.
- **index.html:** Loaded in the Google Fonts (Inter and Space Grotesk) and set up the root div.
- **src/main.jsx:** The entry point where I mount the React App to the DOM.

Code Walkthrough

The App Component (JSX in Action)

In `App.jsx`, I used several powerful JSX features:



- **State Hooks:** Used `useState` and `useEffect` to track scroll position and add a 'scrolled' class to the navbar for a cool glass effect when scrolling down.
- **Array Mapping:** Instead of hardcoding the navigation links, stats, tech stack, and social links, I created arrays of objects and used `.map()` to render them. This makes the code so much cleaner and easier to update! For instance, adding a new skill to the tech stack is just adding one line to the array.
- **Dynamic Styling:** I used inline styles in JSX to set dynamic animation delays for the stats cards (`style={{ animationDelay: `${index * 0.1}s` }}`) and custom background colors for the tech stack dots.

CSS and Styling

I stuck to vanilla CSS because I wanted total control over the design system:

- **CSS Variables:** Defined a strict color palette in `:root` to keep everything consistent.
- **Glassmorphism:** Used `backdrop-filter: blur(12px)` paired with semi-transparent backgrounds to create those sleek, modern cards.
- **Animations:** Created a custom CSS keyframe animation for the floating particles in the background. I generated 20 particles using a quick array map in JSX and gave each a unique class to randomize their position and speed via CSS.

Key Learnings

Setting up the Vite environment manually taught me a lot about modern build tools. It's actually not that scary once you see the individual files. I also solidified my understanding of how JSX compiles down to JavaScript. It's so satisfying to write logic and UI in the same file without context switching.

Challenges and Solutions

One tricky part was getting the typewriter animation to work correctly while keeping the text responsive. I had to play around with the `ch` units and `steps()` in the CSS animation, but eventually found a setup that looks smooth and professional.

Conclusion

I'm really proud of how this landing page turned out. It feels performant, looks great, and the codebase is highly maintainable thanks to React and JSX. I can't wait to tackle tomorrow's challenge!

THANK YOU!